

Examen de fin de semestre SRCS Master 1 SAR

Jonathan Lejeune

Mai 2023

2 heures – **Tout document papier interdit**
Tout appareil électronique interdit

- Le barème est sur 21 points et est donné à titre indicatif. La note finale sera bornée à 20 points.
- Dans le code java demandé, vous ne gérerez pas les imports ni les exceptions **sauf si c'est explicitement demandé**
- Soyez synthétique et précis dans vos réponses
- Il vous est conseillé de lire attentivement les exercices entièrement avant de composer.
- Lorsque vous ne savez pas répondre à une question, il est préférable de s'abstenir que de répondre une absurdité.
- Dans une question de programmation, n'en faites pas plus que ce qui est demandé (vous n'aurez pas de point en plus).
- Une annexe sur l'API JCA/JCE est fournie en fin de document

Exercice 1 – Questions de cours (7 points)

Question 1 1/2 point

Quel est l'intérêt d'avoir un DSL (Domain Specific Language) pour définir des messages et des services en Protobuf ?

Question 2 1 point

Bien que le code suivant compile, pourquoi n'a-t-il pas de sens ?

```
public class Foo implements Remote{ ... }
```

Question 3 1/2 point

En RMI, quels objets représentent une référence distante ?

Question 4 1 point

Pour quelle(s) raison(s) une `MarshallException` peut-elle être jetée ?

Question 5 1 point

À quoi sert un serveur d'application ?

Question 6 1 point

Qu'est ce qu'un EJB stateful ?

Question 7 1 point

Qu'est ce qu'une attaque par rejeu ? Comment peut-on s'en prémunir ?

Question 8 1 point

Qu'est ce qu'une attaque de l'homme du milieu ? Comment peut-on s'en prémunir ?

Exercice 2 – Implantation d'un algorithme distribué (7 points)

Dans cet exercice nous souhaitons implanter en java RMI une variante de l'algorithme de Naimi-Tréhel qui est un algorithme distribué d'exclusion mutuelle. Pour rappel, un algorithme d'exclusion mutuelle offre deux méthodes **lock** et **unlock** permettant respectivement de verrouiller et de déverrouiller une section critique. Dans un environnement distribué chaque processus souhaitant utilisé le verrou est déployé sur un site dédié et la synchronisation entre processus ne se fait que par passage de messages (pas de mémoire partagée). On considérera qu'il n'existe qu'un seul processus par site ce qui fait que les termes "site" et "processus" sont interchangeables. La communication se fait de manière asynchrone, c'est à dire que **l'envoi d'un message n'est pas bloquant**.

L'algorithme se base sur la circulation d'un unique jeton dans le système. La possession du jeton donne le droit exclusif d'entrer en section critique. Les processus sont liés logiquement par un arbre dynamique. La figure 1 donne le pseudo-code de l'algorithme et la figure 2 en donne un exemple d'exécution.

```

1  Local variables :
2  begin
3       $state \in \{tranquil, requesting, inCS\};$ 
4       $next$  : FIFO queue of sites;
5       $last$  : site  $\in \Pi$  or  $nil$ ;

6  Initialization
7  begin
8       $state \leftarrow tranquil;$ 
9       $next \leftarrow \emptyset;$ 
10     if  $self$  is the initial root then
11          $last \leftarrow nil;$ 
12     else
13          $last \leftarrow id$  of initial root;

14 lock()
15 begin
16      $state \leftarrow requesting;$ 
17     if  $last \neq nil$  then
18         send Request( $self$ ) to  $last$ ;
19          $last \leftarrow nil;$ 
20     wait( $state = inCS$ );
21      $state \leftarrow inCS$ ;

22 unlock()
23 begin
24      $state \leftarrow tranquil;$ 
25     if  $next \neq \emptyset$  then
26          $last \leftarrow lastelement(next);$ 
27          $site\ next\_holder \leftarrow dequeue(next);$ 
28         send Token( $next$ ) to  $next\_holder$ ;
29          $next \leftarrow \emptyset$ ;

30 Receive_Request(requester : site ) from site  $s_j$ 
31 begin
32     if  $last = nil$  then
33         if  $state \neq tranquil$  then
34             add requester in  $next$ ;
35         else
36             send Token( $\emptyset$ ) to requester;
37              $last \leftarrow requester$ ;
38     else
39         send Request(requester) to  $last$ ;
40          $last \leftarrow requester$ ;

41 Receive_Token( $q$  : FIFO queue of sites ) from site  $s_j$ 
42 begin
43      $state \leftarrow inCS$ ;
44      $next \leftarrow q + next$ ;

```

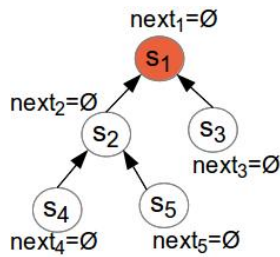
FIGURE 1 – Algorithme de Naimi-Tréhel

Il existe deux types de messages :

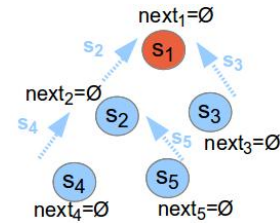
- les messages **request** qui véhiculent l'identifiant du processus demandeur de la section critique
- les messages **token** qui représentent la circulation du jeton. Le jeton contient une FIFO des requêtes en attente de recevoir le droit de rentrer en section critique.

Chaque processus maintient trois variables :

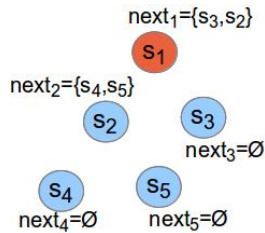
- $state$ son état par rapport à la section critique ($tranquil$ =ne demande pas la CS, $requesting$ = en attente de la CS, $inCS$ = execute la section critique)
- $next$ une file d'attente FIFO d'identifiants de processus demandant la section critique
- $last$ une référence vers le dernier processus qui a demandé la section critique



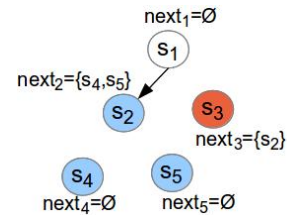
Étape 0 : s_1 possède le jeton et est en section critique. Les autres sites sont à l'état tranquille et leur lien *last* pointe vers un processus parent.



Étape 1 : s_2, s_3, s_4 , et s_5 demandent la section critique et envoient un message *request* à leur lien *last*. Tous les sites sont désormais racines.



Étape 2 : s_1 reçoit la requête de s_3 puis la requête de s_2 . s_2 reçoit la requête de s_4 puis la requête de s_5 .



Étape 3 : s_1 sort de section critique et envoie le jeton à s_3 . s_3 reçoit le jeton et entre en section critique. Le lien *last* de s_1 pointe sur le dernier élément de file *next* du jeton (s_2).

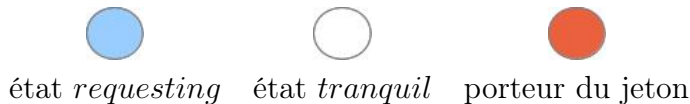


FIGURE 2 – Exemple d'exécution de l'algorithme de Naimi-Tréhel

Question 1 1 point

D'un point de vue RMI, l'état de chaque processus est ainsi représenté par un objet distant. L'identifiant d'un processus est donc une référence distante d'un objet. Écrire l'interface distante `NTNode` qui définit exclusivement les méthodes distantes que chaque processus doit offrir aux autres pour que l'algorithme fonctionne.

Question 2 4 points

Écrire la classe `NTNodeImpl` qui implante l'algorithme de Naimi-Trehel. Pour l'initialisation, la classe offrira deux constructeurs : un constructeur sans argument qui initialise l'attribut `last` à `null` (cas de la racine initiale) et un constructeur qui prend en paramètre la référence vers la racine initiale pour l'affecter à l'attribut `last` (cas des autres processus). Vous ferez en sorte à ce que les états du processus restent cohérents.

Question 3 2 points

Écrire un programme java `Main` qui prend sur sa ligne de commande un booléen (`"true"` ou `"false"`). La valeur du booléen permet d'indiquer si on déploie le processus racine initial ou non. Si le booléen est vrai il faut instancier un `NTNodeImpl`, le transformer en objet distant et l'enregistrer au nom de `"initialroot"` sur un service registry distant supposé requérable sur la machine `"annuaire"` et sur le port 4242. Si le booléen est faux, il est nécessaire de récupérer la référence de l'objet du processus racine initiale via le registry, instancier un `NTNodeImpl` localement et le transformer également en objet distant. Votre code devra assurer que la lecture du registry donnera bien à terme la référence sur la racine initiale (il faut donc prendre en compte le cas où la racine initiale n'a pas encore enregistré sa référence). Pour rappel la méthode `lookup` jette une `NotBoundException` si l'entrée demandée dans le registry n'existe pas.

Question 4 1 point

Pourquoi n'est-il pas pertinent d'implanter un algorithme basé sur le passage de messages avec RMI ?

Question 5 1 point

Dans quel(s) cas de figure, la panne d'un site n'a pas d'impact sur l'exécution de cet algorithme ?

Exercice 3 – Service de certification (5 points)

Dans cet exercice, nous allons implanter l'interface suivante qui représente les fonctionnalités offertes par une autorité de certification. Pour rappel, un certificat est un objet qui permet d'authentifier l'association d'une clé publique avec son propriétaire.

```

1 public interface CertificationAuthority extends Remote{
2
3     /** renvoie la clé publique de l'autorité */
4     PublicKey getPublicKey() throws RemoteException;
5
6     /** renvoie le certificat associé à l'identifiant passé en paramètre,
7      * si le certificat associé à cet identifiant n'existe pas, la méthode
8      * renvoie la référence nulle. */
9     Certif getCertificate(String identifiant) throws RemoteException;
10
11    /** forge et stocke un nouveau certificat associant un identifiant à sa clé
12     * publique. On interdit le fait qu'un identifiant existant puisse changer sa
13     * clé publique via cette méthode. Ainsi, si l'autorité a déjà une clé publique
14     * pour cet identifiant, une RemoteException sera jetée. */
15    Certif declarePublicKey(String identifiant, PublicKey pubk) throws RemoteException;
16 }
    
```

La classe **Certif** modélise un certificat et possède

- 4 attributs constants : une **String** représentant l'identifiant du propriétaire de la clé, une **PublicKey** représentant la clé publique à certifier, un tableau java d'octets qui représente la signature de l'autorité de certification et enfin une **String** indiquant le nom de l'algorithme de signature.
- un constructeur initialisant les 4 attributs via ses paramètres
- un getter pour chaque attribut
- une méthode **boolean** `verify(PublicKey publickeyauthority)` **throws** **GeneralSecurityException** qui vérifie si le certificat est authentique en vérifiant la signature grâce à la clé publique de l'autorité passée en paramètre (la même qui est censée avoir forgé le certificat).

Question 1 2 points

Donner le code de la classe **Certif**. Pour gagner du temps il n'est pas demandé d'écrire le constructeur et les getters. Juste l'entête, les attributs et la méthode **verify** sont demandés.

Question 2 3 points

Donner le code de la classe **CAImpl** qui implante l'interface **CertificationAuthority**. Elle définit en plus un constructeur prenant un nom d'algorithme de chiffrement asymétrique (**String**), une taille de clé (**int**) et un nom d'algorithme de signature (**String**). Ce constructeur crée la paire de clés publique/privée de l'autorité.

Annexe : Type JCA et JCE et principales fonctions

Clés

```
1 public interface Key extends Serializable
2     public String getAlgorithm();
3     public byte[] getEncoded();
4
5 public interface PublicKey extends Key
6 public interface PrivateKey extends Key
7 public interface SecretKey extends Key
8
9 public final class KeyPair implements Serializable
10     public PublicKey getPublic();
11     public PrivateKey getPrivate();
```

Générateur de clés

```
1 public class KeyGenerator
2     public void init(int keysize);
3     public SecretKey generateKey()
4 //Construction d'un KeyGenerator
5 KeyGenerator keyGen = KeyGenerator.getInstance("nom d'algo chiffrement symetrique");
```

```
1 public abstract class KeyPairGenerator
2     public void initialize(int keysize);
3     public KeyPair genKeyPair();
4
5 //Construction d'un KeyPairGenerator
6 KeyPairGenerator kpGen = KeyPairGenerator.getInstance("nom d'algo chiffrement asymetrique");
```

Chiffrement/déchiffrement

```
1 public class Cipher
2     public void init(int opmode, Key key);
3     //opmode=Cipher.ENCRYPT_MODE ou Cipher.DECRYPT_MODE
4     public final byte[] update(byte[] input);
5     public final byte[] doFinal();
6     public final byte[] doFinal(byte[] input);
7 //construction d'un Cipher
8 Cipher cipher = Cipher.getInstance("algo de chiffrement");
```

Hachage sécurisé

```
1 public abstract class MessageDigest
2     public void update(byte[] input);
3     public byte[] digest();
4 //Construction d'un MessageDigest
5 MessageDigest md = MessageDigest.getInstance("algo de hachage");
```

Signature numérique

```
1 public abstract class Signature
2     public void initSign(PrivateKey privateKey);
3     public void initVerify(PublicKey publicKey);
4     public void update(byte[] data);
5     public boolean verify(byte[] signature);
6     public byte[] sign();
7 //Construction d'une Signature
8 Signature signature = Signature.getInstance("algorithme de signature (algo asym+algo hachage)");
```